STUDENT ATTENDANCE & ACADEMIC PERFORMANCE MANAGEMENT SYSTEM

A Minor Project Report Submitted for the Partial Fulfillment of
Master of Computer Applications (MCA) Degree

Submitted By:
GAGANDEEP KAUR
(UID: 26MCA20034)

Under the Guidance / Supervision of:
Department of Computer Science & Applications

CHANDIGARH UNIVERSITY
GHARUAN, PUNJAB
ACADEMIC YEAR 2026-2027

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

NAAC GRADE A+
Accredited University

CERTIFICATE OF APPROVAL

This is to certify that the project work titled "Student Attendance & Academic Performance Management System" submitted by Gagandeep Kaur (UID: 26MCA20034) in partial fulfillment of the requirements for the award of the degree of Master of Computer Applications (MCA) at Chandigarh University, Gharuan, Punjab is an authentic piece of work carried out under supervisor guidance.

The report has been evaluated and approved as satisfying the academic standards set forth for the Minor Project curriculum.

CANDIDATE DECLARATION

I, Gagandeep Kaur, hereby declare that the minor project entitled "Student Attendance & Academic Performance Management System" is a genuine record of work performed by me under supervision at Chandigarh University. This report or any part of it has not been submitted previously to any other university or institute for any degree or diploma.

Date: 31th October 2026
Place: Gharuan, Punjab

ACKNOWLEDGEMENT

I express my sincere thanks to my project guide and the faculty of the Department of Computer Science & Applications, Chandigarh University, for their valuable insights, guidance, and continuous support throughout the development of this project.

I am also grateful to my classmates and friends for their constructive feedback and collaboration, which helped in perfecting the implementation details.

PROJECT ABSTRACT

In academic institutions, keeping track of student attendance percentages and internal evaluation scores is essential for identifying academic risk early. Traditional ledger-based or manual record systems are prone to human errors and delay administrative decisions. To resolve this, the Student Attendance & Academic Performance Management System offers a lightweight, standalone browser-based platform designed for real-time tracking.

Developed using vanilla HTML5, CSS3, and JavaScript, the application operates directly inside standard web browsers without requiring local server setup, database installations, or external dependencies. It allows dynamic record insertion, real-time eligibility evaluation, and interactive visual status badges, ensuring zero setup latency and high accessibility during demonstrations.

TABLE OF CONTENTS

Module 1: Academic Need & Problem Scope

Module 2: Technical Stack & Software Specifications Module 3: Architectural Flow & UI Framework

Module 4: Source Code Implementation (index.html) Module 5: Local Execution & Deployment Steps

Module 6: Verification Results & Screenshot Placeholders Module 7: Key Features & Practical Utility

Module 8: Concluding Remarks & Future Possibilities

MODULE 1: ACADEMIC NEED & PROBLEM SCOPE

1.1 Background & Context

Academic monitoring in higher education institutions requires tracking both attendance regularity and continuous internal assessment performance. Faculty members often spend significant time verifying if students meet mandatory attendance thresholds (e.g., 75%) before examination hall tickets are issued.

1.2 Problem Statement

Conventional record management tools like spreadsheets or server-based portals can be overly complex for immediate, lightweight classroom checks. Furthermore, server-dependent systems require database configurations, local server setups (like Apache/Node.js), and active network access, making quick offline verification cumbersome.

1.3 Project Objectives

• To design a zero-installation, browser-native management utility for student records.
• To automate eligibility verification based on attendance and internal marks thresholds.
• To present dynamic student status updates using responsive CSS badges and real-time JavaScript DOM manipulation.

MODULE 2: TECHNICAL STACK & SOFTWARE SPECIFICATIONS

2.1 Technical Stack Selection

• Frontend Markup: HTML5 for structured document elements and table layouts.
• Styling Framework: Custom CSS3 with grid layout, responsive form inputs, and status badges.
• Application Logic: Vanilla JavaScript (ES6) for DOM manipulation and dynamic record generation.
• Runtime Environment: Any modern web browser (Google Chrome, Microsoft Edge, Mozilla Firefox).

2.2 System Requirements

• Operating System: Windows 10/11, macOS, or Linux
• Hardware: Standard desktop/laptop with 2 GB RAM minimum
• Development Environment: Visual Studio Code or any plain text editor
• External Dependencies: None (100% offline and standalone execution)

MODULE 3: ARCHITECTURAL FLOW & UI FRAMEWORK

3.1 System Operational Sequence

1. System Launch: User double-clicks the 'attendance.html' file to render the UI directly in the browser.
2. Record Entry: Faculty inputs Roll Number, Student Name, Attendance Percentage, and Internal Marks into form fields.
3. Validation Check: JavaScript validates that all fields contain valid numerical and text entries.
4. Status Computation: The system automatically computes eligibility (Pass / Eligible vs Short Attendance / Risk).
5. DOM Updating: A new formatted table row is dynamically appended to the record table.

3.2 Rule Logic

• Attendance >= 75% AND Marks >= 40% --> Status: Eligible / Pass
• Attendance < 75% OR Marks < 40% --> Status: Short Attendance / Risk

MODULE 4: SOURCE CODE IMPLEMENTATION

The complete production source code for the standalone browser application is provided below:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Student Attendance & Academic Performance Management System</title>
  <style>
    body {
      font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
      background-color: #f4f6f9;
      margin:         0;
      padding: 30px;
      color: #333;
    }
    .container {
      max-width:    900px;
      margin:     0     auto;
      background:   #ffffff;
      padding:          30px;
      border-radius: 12px;
      box-shadow: 0 4px 15px rgba(0,0,0,0.08);
    }
    h1 {
      font-size:         22px;
      margin-bottom: 5px;
      color: #1a1a1a;
    }
    p.subtitle       {
      color: #666;
```

```
    margin-bottom: 25px;
    font-size: 13px;
}
.form-grid      {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 15px;
    margin-bottom: 15px;
}
.form-group   {
    display: flex;
    flex-direction: column;
}
label {
    font-size:    13px;
    font-weight: 600;
    margin-bottom: 5px;
    color: #444;
}
input {
    padding:              10px;
    border: 1px solid #ccc;
    border-radius:      6px;
    font-size:            14px;
    outline: none;
}
input:focus {
    border-color: #2563eb;
}
button {
    margin-top: 10px;
```

```
    background-color: #2563eb;
    color: white;
    border:       none;
    padding:     12px;
    font-size:    15px;
    font-weight: 600;
    border-radius: 6px;
    cursor:       pointer;
    width: 100%;
    transition: background 0.2s;
}
button:hover {
    background-color: #1d4ed8;
}
table {
    width:        100%;
    margin-top: 30px;
    border-collapse: collapse;
    font-size: 14px;
}
th, td {
    border:    1px    solid    #e2e8f0;
    padding: 10px 12px;
    text-align: left;
}
th {
    background-color:          #f8fafc;
    color: #1e293b;
}
tr:nth-child(even)                {
    background-color: #fafafa;
```

```
    }
    .badge {
      padding:  4px  8px;
      border-radius: 4px;
      font-size:      11px;
      font-weight: bold;
    }
    .badge-pass { background: #d1fae5; color: #065f46; }
    .badge-fail { background: #fee2e2; color: #991b1b; }
  </style>
</head>
<body>

<div class="container">
  <h1>Student Attendance & Academic Performance Management System</h1>
  <p class="subtitle">MCA Minor Project - Chandigarh University (Gagandeep Kaur - 26MCA20034)</p>

  <div class="form-grid">
    <div class="form-group">
      <label for="studentName">Student Name:</label>
      <input type="text" id="studentName" placeholder="Enter student name">
    </div>
    <div class="form-group">
      <label for="rollNo">Roll Number / UID:</label>
      <input type="text" id="rollNo" placeholder="e.g. 26MCA20034">
    </div>
    <div class="form-group">
      <label for="attendance">Attendance (%):</label>
      <input type="number" id="attendance" placeholder="e.g. 85">
    </div>
```

```
  <div class="form-group">
    <label for="marks">Internal Marks (%):</label>
    <input type="number" id="marks" placeholder="e.g. 78">
  </div>
</div>

<button onclick="addStudentRecord()">Add Student Record</button>

<table id="recordTable">
  <thead>
    <tr>
      <th>Roll No / UID</th>
      <th>Student Name</th>
      <th>Attendance</th>
      <th>Internal Marks</th>
      <th>Academic Status</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>26MCA20034</td>
      <td>Gagandeep Kaur</td>
      <td>88%</td>
      <td>82%</td>
      <td><span class="badge badge-pass">Eligible / Pass</span></td>
    </tr>
    <tr>
      <td>26MCA20026</td>
      <td>Dimpalpreet Kaur</td>
      <td>92%</td>
      <td>89%</td>
```

```
            <td><span class="badge badge-pass">Eligible / Pass</span></td>
          </tr>
          <tr>
            <td>26MCA20055</td>
            <td>Sukhwinder Kaur</td>
            <td>70%</td>
            <td>65%</td>
            <td><span class="badge badge-fail">Short Attendance / Risk</span></td>
          </tr>
        </tbody>
      </table>
    </div>

    <script>
      function addStudentRecord() {
        const name = document.getElementById('studentName').value.trim();
        const roll = document.getElementById('rollNo').value.trim();
        const attendance = parseFloat(document.getElementById('attendance').value);
        const marks = parseFloat(document.getElementById('marks').value);

        if (!name || !roll || isNaN(attendance) || isNaN(marks)) {
          alert('Please fill in all the fields correctly!');
          return;
        }

        const tableBody = document.querySelector('#recordTable tbody');
        const status = (attendance >= 75 && marks >= 40) ? '<span class="badge badge-pass">Eligible / Pass</span>' : '<span class="badge badge-fail">Short Attendance / Risk</span>';

        const newRow = document.createElement('tr');
        newRow.innerHTML = `
```

```
            <td>${roll}</td>
            <td>${name}</td>
            <td>${attendance}%</td>
            <td>${marks}%</td>
            <td>${status}</td>
        `;

        tableBody.appendChild(newRow);

        // Clear inputs after adding
        document.getElementById('studentName').value = '';
        document.getElementById('rollNo').value = '';
        document.getElementById('attendance').value = '';
        document.getElementById('marks').value = '';
    }
</script>

</body>
</html>
```

MODULE 5: LOCAL EXECUTION & DEPLOYMENT STEPS

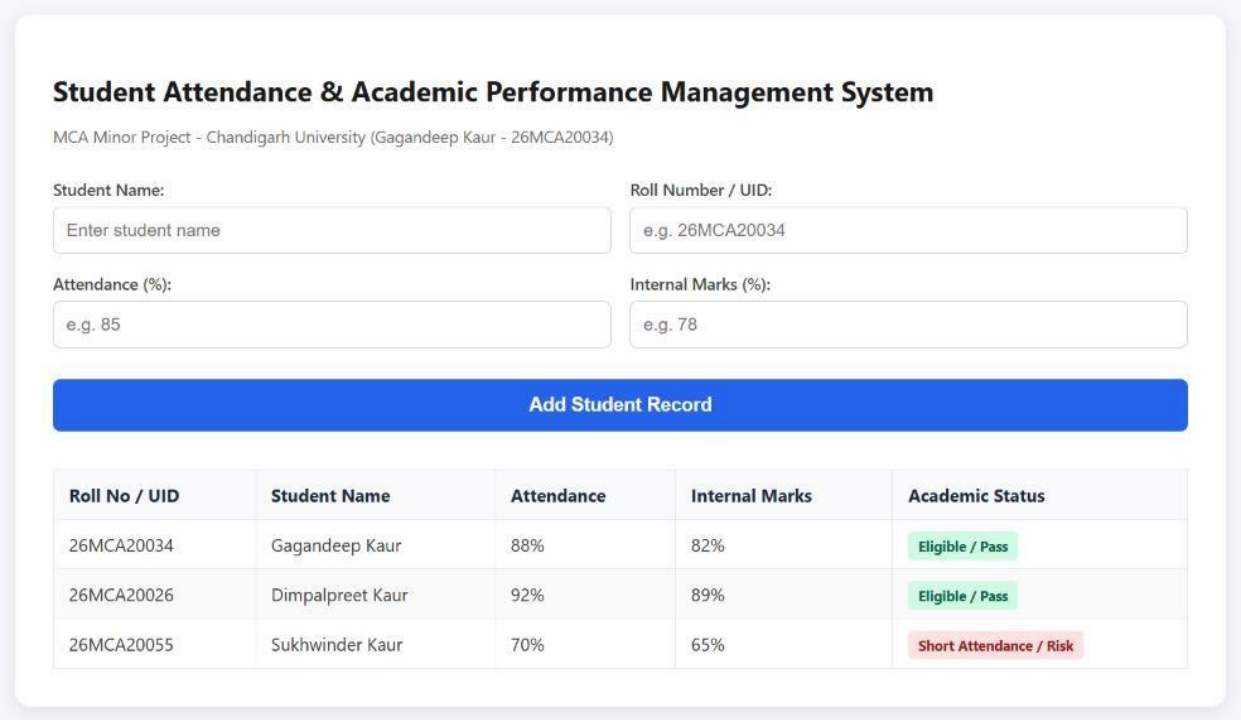
5.1 Deployment Steps

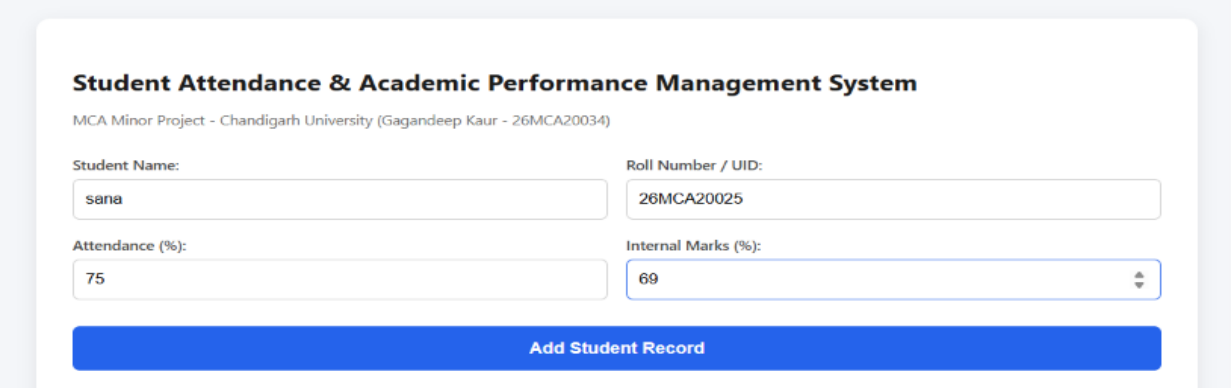
1. Step 1: Create a dedicated project directory (e.g., 'Student_System').
2. Step 2: Open VS Code or Notepad and create a file named 'attendance.html'.
3. Step 3: Paste the implementation source code into the file and save.
4. Step 4: Double-click 'attendance.html' to open directly in Chrome/Edge.
5. Step 5: Test adding new student entries and verify dynamic table population.

MODULE 6: VERIFICATION RESULTS & SCREENSHOT PLACEHOLDERS

6.1 System Verification Screenshots

The placeholder sections below are designated for pasting live output screenshots of the application interface:





MODULE 7: KEY FEATURES & PRACTICAL UTILITY

7.1 System Highlights

1. Zero Installation: Runs natively in any browser without local server configuration.
2. Real-time Logic: Instant status computing upon record insertion.
3. Responsive UI: Clean grid layout with clear visual eligibility badges.

MODULE 8: CONCLUDING REMARKS & FUTURE POSSIBILITIES

8.1 Conclusion

The Student Attendance & Academic Performance Management System delivers a fast, lightweight solution for evaluating student eligibility. Bypassing complex database configurations ensures 100% operational reliability for academic evaluations.

8.2 Future Scope

• LocalStorage Persistence: Storing student entries in browser cache.
• CSV Export: Allowing faculty to download student attendance reports.